



Recycling Robot Documentation

Version 1.0.0

Rimvydas Kersys

March 17, 2026

Contents

1	Project Overview	3
2	System Architecture	3
2.1	Hardware Architecture	4
2.2	Software Architecture	5
3	Hardware Setup	5
3.1	Robot	5
3.2	Raspberry Pi	6
3.2.1	Hardware Used	6
3.2.2	Camera Positioning	6
3.2.3	Motor Driver Considerations	7
3.3	Gripper & Motor Driver	7
4	Communication & Networking	7
4.1	Network Architecture	7
4.2	IP addressing	8
4.3	Communication	8
5	System Service	8
5.1	Runtime File Protection	9
6	Robot Motion & Control	9
6.1	Coordinate Mapping	9
6.2	Motion Sequence	9
7	Safety Considerations	10
7.1	Room Specific Safety Measures	10
7.2	Robot Safety	10
7.3	Electrical Safety	11
7.4	Mechanical Safety	11
8	Setup & Deployment Guide	11
8.1	Raspberry Pi	11
8.1.1	Imaging the Raspberry Pi	11
8.1.2	Connecting via SSH	11
8.1.3	Deployment	12
8.2	Server	13

8.3	Start-up Instructions	13
8.4	Power-down Instructions	14
9	Debugging & Troubleshooting	15
10	Limitations & Known Issues	15
11	Future Work	15

List of Code Listings

1	"verifying the dnsmasq service"	8
2	"Updating Raspberry Pi code	9
3	"How to SSH into a Raspberry Pi"	11
4	"Finding the IP address of a Raspberry Pi via terminal on personal device"	12
5	"Finding the IP address of a Raspberry Pi via on device terminal"	12
6	"Raspberry Pi Deployment"	12
7	"Restarting the Raspberry Pi Service"	12
8	"Server Deployment"	13
9	"Checking and Restarting the DNS service on Server machine"	13
10	"Running Server code"	14

1 Project Overview

This project implements an autonomous waste-sorting system using a KUKA KR180-2 industrial robot, a vision-based detection pipeline, and a Raspberry Pi-controlled motorised gripper.

The system is designed to detect recyclable objects on a conveyor belt, classify them using a machine learning model, and command the robot to pick and place each item into the correct bin.

System Workflow:

1. A complete sorting cycle operates as follows:
2. The vision model processes a live camera feed and detects waste objects.
3. The detected object's position is transformed into robot coordinates.
4. The robot moves to the target position.
5. The Raspberry Pi actuates the gripper to grasp the object.
6. The robot transports the object to the appropriate disposal bin.
7. The robot returns to its home position and awaits the next detection.

The system is modular. Vision processing, robot motion, and gripper control operate as independent components communicating over a local network.

2 System Architecture

2.1 Hardware Architecture

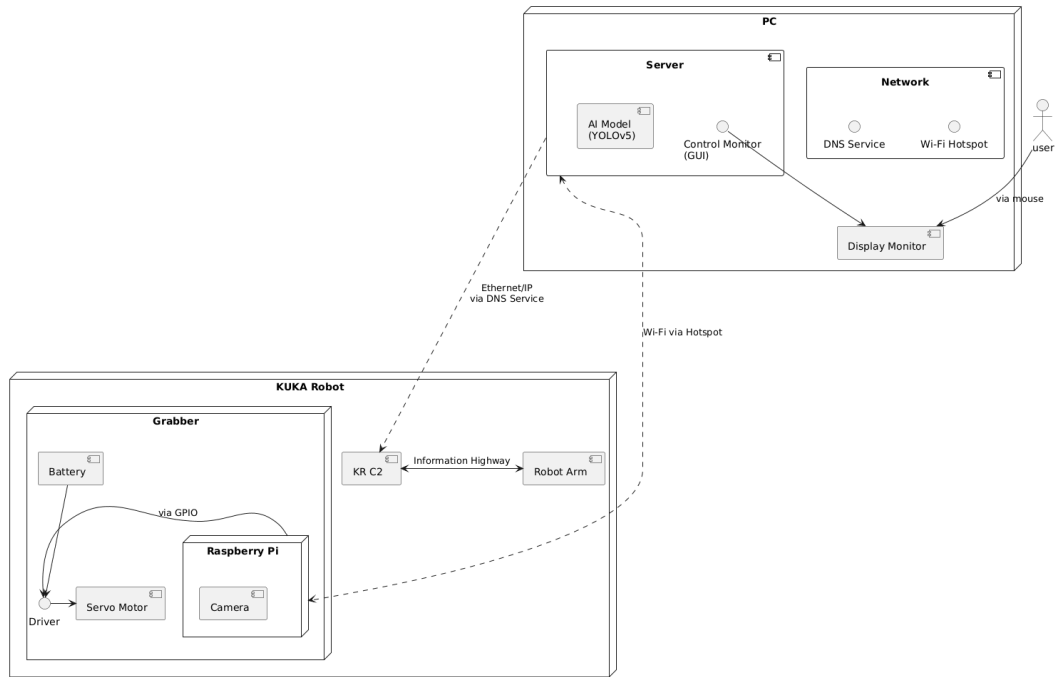


Figure 1: Full Hardware Component Diagram

2.2 Software Architecture

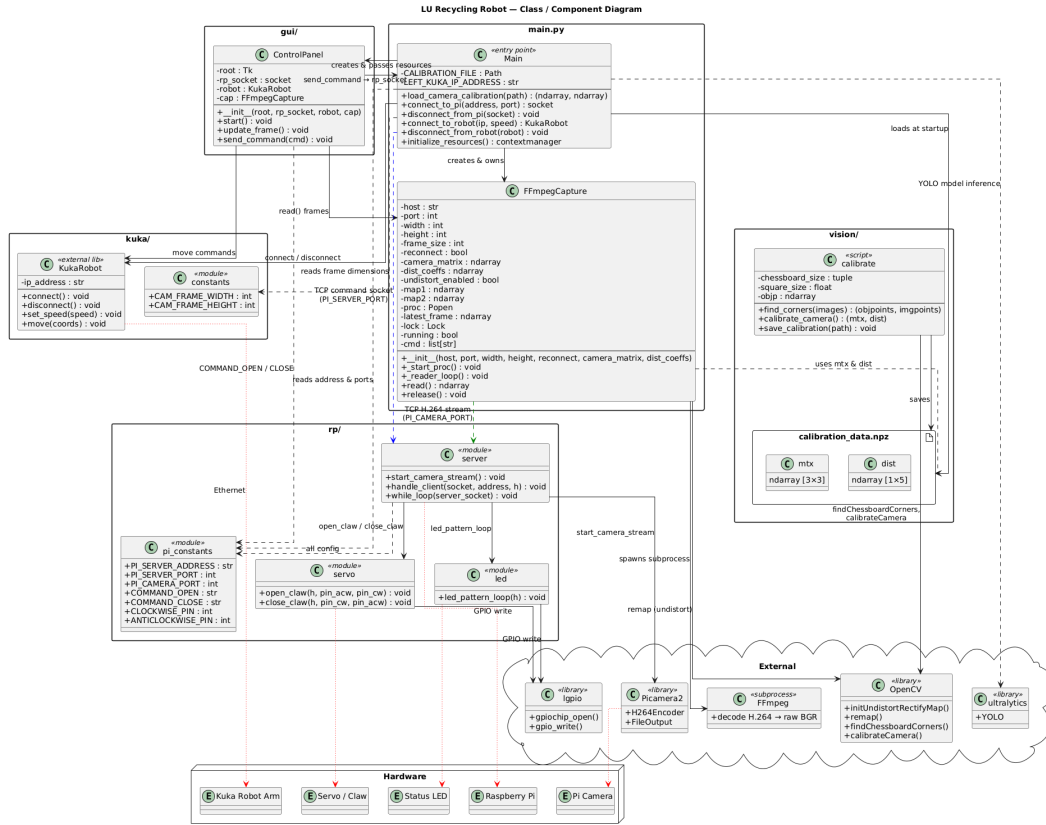


Figure 2: Full Software Component Diagram

3 Hardware Setup

3.1 Robot

The robot position is based on the position of its tool. Depending on the grabber size and angle, you may want to calibrate the robot to these. It is important to note that the robot is not entirely accurate and may still go to a location which is off the intended location by up to 2-3mm or more if these errors are related to the angle of the tool.

The robot can also be calibrated (mastered) to help alleviate some location inconsistencies however this is out of scope for this document.

Currently, the robot's position is not absolute. The robot is visually not angled correctly. Despite this, absolute location errors are accounted for via software.

3.2 Raspberry Pi

The Raspberry Pi acts as the embedded controller for the:

- PiCamera Module 3
- L298N motor driver
- Gripper motor

3.2.1 Hardware Used

- Raspberry Pi 5¹
- PiCamera Module 3
- L298N DC Motor Driver
- 12V Battery
- Portable Charger (Pi power supply)
- 3-6V DC Motor (gripper actuation)

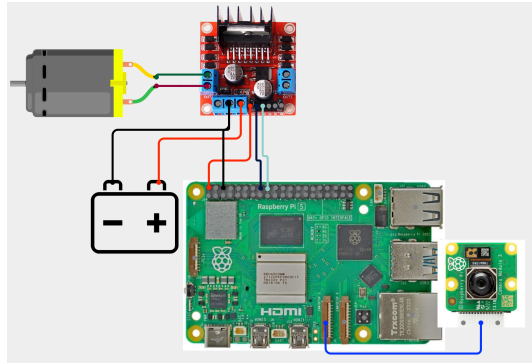


Figure 3: Components wired together

3.2.2 Camera Positioning

The PiCamera is mounted beneath the robot arm and positioned approximately perpendicular to the conveyor belt. This orientation minimises perspective distortion and simplifies coordinate transformation in software.

If mounted facing downwards, autofocus inconsistencies may occur. Focus correction should be handled in software or manually adjusted if using older camera modules.

¹While a Raspberry Pi 5 is used in this setup, lower-power models (like the Raspberry Pi Zero) may be used if performance requirements permit.

3.2.3 Motor Driver Considerations

The L298N introduces an approximate 2V voltage drop between supply and motor output.

For this reason:

- A 6V battery is insufficient for long-term operation
- A 12 V batter is recommended
- The resulting motor voltage (10V after drop) provides:
 - Faster actuation (2s vs 5s)
 - Improved torque margin
 - More reliable gripping under load

If variable motor speed control is required, the enable (ENA) pin may be connected to a PWM-capable GPIO pin instead of the fixed 5V pin as in Fig 3.

3.3 Gripper & Motor Driver

The Motor and Driver are both affixed to the arm such that the motor rotates a bolt perpendicular to the arm.

4 Communication & Networking

The system operates over a local Wi-Fi network hosted by the server machine

4.1 Network Architecture

- The server broadcasts a hotspot with:
 - SSID: kukaman
 - Password: kukawifi
- The Raspberry Pi connects automatically to this network.
- The robot controller communicates with the server over Ethernet

4.2 IP addressing

The dnsmasq service is used on the server machine to

- Assign IP addresses to the two KR C2 controllers
- Provide lightweight DHCP functionality
- Maintain stable local addressing

To verify the service is running, run

Listing 1: "verifying the dnsmasq service"

```
$ journalctl -xefu dnsmasq
```

in the server machine's terminal.

4.3 Communication

1. Vision detection runs on the server
2. Coordinates of object are calculated
3. Commands are sent to the robot controller
4. Gripper commands are sent to the Raspberry Pi
5. Any relevant status messages are logged for debugging

All communication occurs within the local network and does not require internet access.

5 System Service

The Raspberry Pi runs a "systemd" service called `rpi-server.service` to ensure

- Automatic startup on boot
- Controlled deployment on runtime files
- Automatic restart on failure

5.1 Runtime File Protection

On service start:

1. A **/local** directory is created
2. All files from /rp are copied into **/local**
3. The program runs from **/local**

This design prevents:

- Runtime corruption during code edits
- Partial updates while the system is running
- Inconsistent execution states

To apply updated code do one of the following:

```
Listing 2: "Updating Raspberry Pi code"
$ systemctl restart rpi-server.service
-----
$ sudo reboot
```

The service is configured to automatically recover in the event of a crash.

6 Robot Motion & Control

The robot operates in tool-based coordinate space.

6.1 Coordinate Mapping

In software, object positions detected by the vision system are:

1. Extracted in image coordinates
2. Converted to real-world measurements
3. Robot coordinates calculated relative to current robot position

6.2 Motion Sequence

For each object detected:

1. Move to safe, pre-grasp, height
2. Open gripper
3. Descend vertically
4. Close Gripper
5. Ascend to safe clearance height
6. Move to correct bin location
7. Release object by opening gripper
8. Close gripper
9. Return to home position

7 Safety Considerations

This system uses an industrial robotic arm and moving mechanical components. Proper safety procedures must always be followed.

To do this, it is essential that operators of the robot complete a risk assessment to ensure that all risks are noticed and procedures are put in place in the event that risks occur.

7.1 Room Specific Safety Measures

In the room in which the robots are based, it is essential to wear steel capped boots and ensure that all foods and drinks are kept in sealed containers.

7.2 Robot Safety

Under no circumstances should the robot be operated whilst another person is inside of the cage with the robot.

When operating in auto or Ext. mode, the Robot will emergency stop if the safety gate is open. However, in both T1 and T2 modes, the robot can still be manually controlled even whilst the gate is open. Ensure that nobody is inside the cage and that the gate is closed before operating the robot.

7.3 Electrical Safety

Under no circumstances should the KR C2 controller be opened while operating. Take necessary precautions as described in the KR C2 manual before opening the controller.

7.4 Mechanical Safety

The gripper creates many pinch points during operation. Before operating the gripper, ensure that all hands are clear of the gripper.

8 Setup & Deployment Guide

8.1 Raspberry Pi

8.1.1 Imaging the Raspberry Pi

First, create a Raspberry Pi OS image for the respective Pi device. Ensure when making the image to enable SSH and set the correct Wi-Fi name and password. During operation, the Server machine broadcasts a Wi-Fi connection via hotspot with SSID: "kukaman" and password: "kukawifi" which should be used when robot is running. However, any other Wi-Fi connection can be used when first initialising.

8.1.2 Connecting via SSH

When using the terminal, there are 2 methods which can be used to connect to the Raspberry Pi.

Listing 3: "How to SSH into a Raspberry Pi"

```
$ ssh {deviceName}@{userName}
# The currently used Raspberry Pi would be kuka@KukaPi

$ ssh {deviceName}@{IPAddress}
# The currently used Raspberry Pi would be kuka@10.42.0.218
```

If using an app to access the Raspberry Pi, such as VNC Viewer. You will likely need to provide the IP address.

To find out the IP address of the Raspberry Pi, you may use

Listing 4: "Finding the IP address of a Raspberry Pi via terminal on personal device"

```
$ arp -a
```

or you may first SSH into it via the terminal using a username and then use

Listing 5: "Finding the IP address of a Raspberry Pi via on device terminal"

```
$ hostname -I
```

which will output the Raspberry Pi's IP address. Or, you may instead connect an external display to the Raspberry Pi and using an external keyboard and mouse hover over the Wi-Fi icon to see the Raspberry Pi' IP address

8.1.3 Deployment

Finally, to get the Raspberry Pi to run the program on start up:

Listing 6: "Raspberry Pi Deployment"

```
$ git clone https://github.com/Rimichu/LU-Recycling-Robot
$ cd LU-Recycling-Robot/rp
```

```
$ # Copy (and if it exists, replace) the service
$ sudo cp -fr rpi-server.service
    /etc/systemd/system/rpi-server.service
```

```
$ # Start Service (May require password)
$ systemctl start rpi-server.service
```

```
$ # Check to ensure that the service is working
$ journalctl -xefu rpi-server.service
```

When the service starts, the service will create a new directory "local" and copy all files from the /rp directory there to be used during runtime. This will allow you to change code in the /rp directory with no danger to the program during run time. Upon changing code, to use the new code just run either

Listing 7: "Restarting the Raspberry Pi Service"

```
$ systemctl restart rpi-server.service # Restarts the service
$ sudo reboot                          # Restarts the Raspberry
```

which will again re-copy all files and run the program.

8.2 Server

Clone the repository, create and open the kuka environment:

Listing 8: "Server Deployment"

```
$ git clone https://github.com/Rimichu/LU-Recycling-Robot
$ cd LU-Recycling-Robot
$ conda env create -f env.yaml
$ conda activate kuka
```

8.3 Start-up Instructions

1. Turn on all necessary devices
 - (a) Turn on the Kuka Controller (KR C2) via Lever and follow any on screen instructions until the Human Machine Interface (HMI) panel appears
 - (b) Ensure KR C2 is set to Auto Mode
 - (c) Start up the server machine (Password: kuka)
 - (d) Place and plug in 12V battery into driver
 - (e) Place portable charger on robot arm ensuring enough clearance for other components
 - (f) Plug in portable charger into raspberry pi
 - (g) Turn off overhead lights
2. On the server machine, ensure that the dnsmasq service is running correctly via the terminal

Listing 9: "Checking and Restarting the DNS service on Server machine"

```
# Check on the service
$ journalctl -xefu dnsmasq
# Use Ctrl+C to continue using the terminal

# Restart the service
$ systemctl restart dnsmasq
```

3. On the KR C2 controller, make yourself administrator ²
 - (a) Configure → User Group → Administrator (Password: kuka)
 - (b) Ensure Num Lock is Disabled → Ctrl+Esc → IssacStartC3Bridge
 - (c) Alt+Tab into the Kuka HMI
4. Inside the HMI, run the C3BI_RUN program
 - (a) Press enter when selected
 - (b) Reduce program speed to a recommended 50%
 - (c) Press the green button once to get robot to its starting position
(current position as default)
 - (d) Press the green button again to begin robot program
5. In the Server machine's terminal:

Listing 10: "Running Server code"

```
$ cd LU-Recycling-Robot # Ignore if already in directory
$ conda activate kuka   # Ignore if already in kuka environment
$ python3 main.py
```

8.4 Power-down Instructions

1. Server Machine
 - (a) Ensure program is off by one of the following in order of safety:
 - i. Quit Safely Button
 - ii. Exiting the program via exit button
 - iii. Using Ctrl+C in the terminal
 - (b) Power off server machine
2. Kuka Robot
 - (a) Press "Stop" button to stop any actively running program.
 - (b) TODO: How to navigate to fresh start turn off

²See Debugging & Troubleshooting if the safety gate was opened whilst the KR C2 was turned on

- (c) Power off Kuka Controller (KR C2) via lever
- 3. Motor Battery
 - (a) Unplug 12V battery
 - (b) take 12V battery off the mount to charge
- 4. Raspberry Pi
 - (a) Unplug portable charger from Raspberry pi
 - (b) Take portable charger off the mount to charge

9 Debugging & Troubleshooting

If the gate is opened whilst the KR C2 is operating, the KR C2 will no longer be able to run code. To fix this, ensure the gate is closed and flip the lever off and on to soft reset the KR C2.

10 Limitations & Known Issues

11 Future Work